

Desafío II – Ude@Book

Preinforme de análisis y diseño

JOSE GABRIEL GIRALDO FLOREZ

Juan Nicolás Rubiano Barreiro

Octubre de 2026

Índice

1. Contexto del problema	2
2. Análisis del problema	2
2.1. Entidades identificadas	2
2.2. Funcionalidades y exigencias	2
2.3. Restricciones para el condicionamiento del diseño	3
3. Solución propuesta	4
3.1. Clases	4
3.2. Relaciones principales	5
3.3. Decisiones de programación orientada a objetos	5
4. Persistencia de los datos	6
5. Medición de recursos	6
6. Plan de implementación	7

1 Contexto del problema

Ude@Book es una red social cuyo propósito es implementar la capa lógica, con interacciones a través de consola. Para el sistema se debe permitir que un usuario inicie sección, administre sus amistades, visite el perfil de sus amigos, reciba sugerencias de amistad e intercambie mensajes. Además, después de cada funcionalidad se hace un reporte de los recursos consumidos.

Por su parte, dejando por fuera el registro de usuarios, la eliminación de cuentas, la creación y edición de álbumes y fotos, la eliminación de amistades y la respuesta directa a mensajes. Dejando para los datos de partida su cargado desde archivos con un formato definido a decisión del equipo, contando con pruebas de al menos 1000 usuarios.

2 Análisis del problema

2.1 Entidades identificadas

Del enunciado se identifica lo siguiente:

- **Usuario:** Nombre único, contraseña, correo, ciudad, país, fecha de nacimiento y fecha de inscripción.
- **Álbum:** Pertenece a un usuario y tiene nombre y privacidad (público, privado o amigos).
- **Foto:** Pertenece a un álbum y tiene nombre, fecha de captura, tamaño y cantidad de likes. (cuenta con una ruta directa, no la implementación).
- **Mensaje:** tiene emisor, receptor, fecha, texto de hasta 512 caracteres y estado de visualización (Leído o no Leído).

A lo cual se suman dos relaciones con reglas propias:

- **La amistad, que es mutua:** Si A es amigo de B, B es amigo de A.
- **La solicitud de amistad, que es dirigida:** tiene un origen y un destino.

2.2 Funcionalidades y exigencias

- **Inicio de sección:** Para el inicio de sección se busca el usuario por nombre y validar contraseña, Procede a mostrar las fotos de perfil de 3 amigos elegidos a lazar, sin repeticiones dentro del grupo. Dicha probabilidad, depende de los

amigos en común: peso 1 o normal si son 10 o menos, 2.5 si son entre 11 y 20, y 3.5 si son más de 20. Con posibilidad de 1 única actualización.

- **Administración de amistades:** Enviar solicitudes y aceptar las recibidas. No se permite el enviar solicitud a quien ya es amigo, a quien ya nos envió una, ni a uno mismo.
- **Visita al perfil de un amigo:** Se muestra solo los álbumes públicos y de amigos, y recorre sus fotos en un carrusel circular con las opciones de siguiente, anterior, dar o quitar like, y salir.
- **Sugerencias de amistad:** Debe mostrarse K sugerencias (entre 1 y 20) entre los amigos de los amigos, ordenadas por la cantidad de amigos en común, con la opción de enviar solicitud.
- **Mensajes:** Enviar a un amigo (sin texto vacío y con un máximo de 512 caracteres), leer los mensajes no leídos y hacer su respectivo cambio de estado, y buscar mensajes para un rango de fecha establecido.
- **Medición de recursos:** Al finalizar cada funcionalidad, mostrar las iteraciones realizadas y la memoria total en uso, incluidas las variables locales y los parámetros por valor (Incluye las veces que se haga uso de alguna librería externa).

2.3 Restricciones para el condicionamiento del diseño

La solución del programa debe realizarse con lenguaje C++ con el uso de programación orientada a objetos dentro de QT con las siguientes restricciones:

- No se debe usar contenedores STL ni estructuras struct.
- No es permitido instrucciones de ANSI C.
- Las estructuras de datos deben ser propias y con el uso de memoria dinámica.
- Los archivos son de uso exclusivo para cargar y guardar información, no se permite el uso de estos como área de procesamiento.
- Solo se permiten bibliotecas externas para la generación de números aleatorios y para captura de fecha y hora del sistema. Toda comparación o validación de fechas es implementación propia del desarrollador.
- La eficiencia es un criterio central para la evaluación. Por eso cada decisión debe ser tomada buscando el menor costo en tiempo y memoria para grandes volúmenes de datos.
- No se usa `std::string`. Los textos (nombres, clave, correo, ciudad, país, nom-

bres de álbumes y fotos, y mensajes) se almacenan como `Lista<char>`, la misma plantilla propia de las colecciones.

- Las operaciones que normalmente resuelven las bibliotecas, como recorrer los archivos, convertir entre texto y número, y comparar textos, son implementación propia.

3 Solución propuesta

3.1 Clases

El diseño cuenta con 9 clases propias y un componente externo:

- **UdeABook:** Clase para representación de la plataforma. Es dueña de todos los usuarios y de la red de amistades, encargada de administrar la sección y el menú, coordina las funcionalidades y se encarga de cargar y guardar los datos referentes al programa. Además, implementa la lectura y escritura de archivos y consola carácter por carácter, la conversión entre texto y número, y la comparación de textos, sin depender de las bibliotecas.
- **Usuario:** Contiene los datos personales, los álbumes, los mensajes enviados y recibidos, y las solicitudes de amistad recibidas asociadas a dicho perfil.
- **Álbum y foto:** Se encargan de modelar el contenido visual. La privacidad del álbum se representa con un tipo enumerado de 3 valores.
- **Mensaje:** Gestiona el guardado del emisor y el receptor como identificadores numéricos, la fecha, el texto y el estado de lectura.
- **Fecha:** Almacena consigo el año, mes, día, hora y minuto. La comparten Usuario, Foto y Mensaje. Permite validar fechas existentes, incluyendo los años bisiestos, y sobrecarga el operador `<` para la comparación de fechas.
- **Lista<T>:** Plantilla propia de arreglo dinámico. Reemplaza los contenedores de la STL e implementa todas las colecciones del sistema. También almacena los textos del sistema como `Lista<char>`, aprovechando su copia profunda, su liberación de memoria y su medición, y obteniendo la longitud de cada texto en tiempo constante. Sobrecarga el operador `[]` para el acceso por posición en tiempo constante.
- **RedAmistad:** Matriz de adyacencia que almacena todas las amistades y resuelve las consultas entre ellas.
- **Métricas:** Lleva el conteo de iteraciones, memoria y uso de componentes ex-

ternos. Sus miembros son estáticos, para la implementación en cualquier clase, permitiendo reportar sin necesidad de un objeto.

- **Random:** Componente externo de la biblioteca `<random>`, usado para el sorteo de las fotos de amigos.

3.2 Relaciones principales

- Cada mensaje existe una sola vez. Es parte del usuario que lo recibe, quien es el responsable de liberarlo. El emisor solo guarda una referencia a él. Así no se duplica información y se evita liberar la misma memoria 2 veces.
- La amistad no se guarda en cada usuario si no en RedAmistad. De lo contrario, cada amistad quedaría con un doble registro.
- Las solicitudes se modelan como una asociación de Usuario consigo mismo. Cada usuario guarda los identificadores de quienes enviaron una solicitud.
- `Lista<T>` es parte de UdeABook, Usuario, Álbum, Foto y Mensaje. Las tres primeras la usan para sus colecciones, y las cinco para sus textos.

3.3 Decisiones de programación orientada a objetos

Encapsulamiento:

- Todos los atributos son de carácter privado y se accede a ellos con getters constantes.
- Las colecciones solo exponen un getter por referencia constante, para evitar el reemplazo desde fuera de la clase.
- La contraseña no cuenta con getter: solo puede verificarse.

Copia:

- Todas las clases cuentan con constructor de copia.
- Aquellas que cuentan con memoria dinámica implementan copia profunda y sobre carga del operador de asignación.

Sobrecarga de métodos:

Se sobrecarga `imprimirRuta`, para la ruta de la foto de perfil y la de una foto del álbum, y `contar` en `Metricas`, para sumar una iteración o varias. Varios constructores usan valores por defecto. En UdeABook se sobrecargan `leer` y `escribir`, con tres versiones cada uno: para textos, números y fechas.

Funciones amigas:

No se usan. Cada clase cuenta con getters suficientes, y declarar funciones amigas habría roto el encapsulamiento sin ninguna ventaja.

4 Persistencia de los datos

La información se almacenará en archivos base: Usuarios, amistades, solicitudes, mensajes y álbumes con fotos. Cada uno de los archivos inicia con la cantidad de registros, lo cual permite reservar la memoria exacta al cargar. Las referencias entre registros se deberán guardar por identificador, haciendo que la carga no necesite hacer búsquedas.

Para evitar el reescribir archivos completos por cada iteración, se usa un archivo de cambios. Cada modificación se agrega al final como una línea corta, en tiempo constante. Esto se dará según su importancia, dejando un guardado en 3 momentos:

Momento	Qué se guarda
De inmediato	Los mensajes enviados y las lecturas de mensajes. Cada lectura se registra en una sola línea, sin importar cuantos mensajes se marcaron como leídos.
Al salir del apartado	Las solicitudes enviadas, las aceptaciones y los likes, en un solo lote. De los likes solo se guarda el valor final.
Al salir del programa	La consolidación: se reescriben únicamente los archivos que tuvieron cambios y se vacía el archivo de cambios.

Cuadro 1: Momentos de guardado de información.

Al iniciar, el programa carga los archivos base y aplica los cambios pendientes. Esto permite recuperar la información si el programa fue cerrado de forma inesperada.

El esquema cumple la restricción del enunciado. Ya que los archivos solo escriben para y guardar y solo se leen para cargar: todo el procesamiento ocurre en memoria.

5 Medición de recursos

Iteraciones. Cada ciclo acumula sus iteraciones en un contador local y lo reporta una sola vez al terminar. También se registra cada uso de un componente externo.

Memoria. En lugar de recorrer todas las estructuras después de cada funcionalidad, lo que costaría tiempo proporcional al total de datos, la clase Metricas mantiene un contador: se actualiza en cada reserva y liberación de memoria dinámica; a ese valor se suma el tamaño de las variables locales y los parámetros por valor de la funcionalidad.

Como los textos se almacenan en `Lista<char>`, su memoria queda incluida automáti-

camente en este conteo.

Con esto, el reporte se obtiene en tiempo constante.

6 Plan de implementación

La implementación se hará módulo por módulo, probando cada clase antes de pasar a la siguiente: Metricas, Fecha, Lista<T>, Foto, Album y Mensaje, Usuario, RedAmistad y finalmente UdeABook con el menú.

En paralelo se construirá un generador de datos de prueba con al menos 1000 usuarios, amistades, mensajes distribuidos en más de seis semanas y álbumes de cinco o más fotos, de forma que cubra las pruebas requeridas para el video.